



PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 3

Introduction to OOPS Programming

SESSION / ASSIGNMENT – 6

TASKS :

1. Create a class called Song in your preferred OOP language with private properties title and artist. Add public getter and setter methods to access and modify these properties, then create an object and update its title.



```
#include <iostream>
#include <string>
using namespace std;

class Song {
private:
    string title;
    string artist;

public:
    // Setter for title
    void setTitle(string title) {
        this->title = title;
    }

    // Getter for title
    string getTitle() {
        return title;
    }

    // Setter for artist
    void setArtist(string artist) {
        this->artist = artist;
    }

    // Getter for artist
    string getArtist() {
        return artist;
    }
}
```

```

};

int main() {
    Song song;

    song.setTitle("Shape of You");
    song.setArtist("Ed Sheeran");

    cout << "Title: " << song.getTitle() << endl;
    cout << "Artist: " << song.getArtist() << endl;

    // Update title
    song.setTitle("Perfect");

    cout << "Updated Title: " << song.getTitle() << endl;

    return 0;
}

```

OUTPUT:

```

Title: Shape of You
Artist: Ed Sheeran
Updated Title: Perfect

```

2. Build a class called `InstaStory` with a protected property `storyViews`. Create a subclass called `SponsoredStory` that can access and display the `storyViews` value.



```

#include <iostream>
using namespace std;

class InstaStory {
protected:
    int storyViews;

public:
    InstaStory(int views) {
        storyViews = views;
    }
};

class SponsoredStory : public InstaStory {
public:
    SponsoredStory(int views) : InstaStory(views) {
    }

    void displayViews() {
        cout << "Story Views: " << storyViews << endl;
    }
}

```

```

    }
};

int main() {
    SponsoredStory story(1500);

    story.displayViews();

    return 0;
}

```

OUTPUT:

```
Story Views: 1500
```

- Imagine a Flipkart-like app: create an abstract class `Product` with an abstract method `upload()`. Then, create two subclasses, `Electronics` and `Clothing`, that each implement the `upload()` method to print a different upload message.



```

#include <iostream>
using namespace std;

// Abstract class
class Product {
public:
    virtual void upload() = 0;
};

// Electronics class
class Electronics : public Product {
public:
    void upload() override {
        cout << "Uploading electronic product details and specifications." << endl;
    }
};

// Clothing class
class Clothing : public Product {
public:
    void upload() override {
        cout << "Uploading clothing product details and size chart." << endl;
    }
};

int main() {
    Electronics e;
    Clothing c;
}

```

```
e.upload();  
c.upload();  
  
return 0;  
}
```

OUTPUT:

```
Uploading electronic product details and specifications.  
Uploading clothing product details and size chart.
```

4. Refactor the following code so that the user's phone number in the UserProfile class is private and can only be set or retrieved using public methods.

Hint: Add private access modifier to the phone number and create setPhoneNumber() and getPhoneNumber() methods.



```
#include <iostream>  
#include <string>  
using namespace std;  
  
class UserProfile {  
private:  
    string phoneNumber;  
  
public:  
    void setPhoneNumber(string phone) {  
        phoneNumber = phone;  
    }  
  
    string getPhoneNumber() {  
        return phoneNumber;  
    }  
};  
  
int main() {  
    UserProfile user;  
  
    user.setPhoneNumber("9876543210");  
  
    cout << "Phone Number: "  
        << user.getPhoneNumber() << endl;  
  
    return 0;  
}
```

OUTPUT:

Phone Number : 9876543210

5. Explain in your own words the difference between encapsulation and abstraction, using examples from any social media app you use daily (like Instagram or WhatsApp).



Encapsulation

Encapsulation means **hiding data and controlling access to it using methods**.

Example (Instagram):

When you update your Instagram password, the password is stored privately inside Instagram's system. You cannot access it directly. Instagram provides options like **Change Password** through which the password can be modified.

This is encapsulation because the data is hidden and accessed only through controlled methods.

Abstraction

Abstraction means **showing only the necessary features and hiding the internal implementation details**.

Example (WhatsApp):

When you send a message on WhatsApp, you simply click the Send button. You do not see how the message is encrypted, transmitted through servers, and delivered to the receiver.

WhatsApp hides all these complex processes and only shows the Send feature to the user.

This is abstraction because the implementation details are hidden from the user.